In the name of God

University of Tehran

Faculty of Electrical and Computer Engineering

Deep Generative Models

Homework 1

Full Name	Mohammad Taha Majlesi Koupai
Student Number	810101504
Submission Date	2024/11/04

Contents

1 Part 1 - PGM

1.1 Subsection 1: Drawing the Bayesian Network

Given the provided explanations and variables, the Bayesian network is drawn as follows:

Variables:

- **A**: The air conditioning system fails.
- **O**: The server room door is left open for a long time.
- **T**: The room temperature exceeds a certain threshold.
- **M**: A text message is received.
- **S**: The warning siren is activated.

Relationships between variables:

- **A** and **O** have a direct effect on **T**.
  - This means **A** and **O** are parents of **T**.
- **T** has a direct effect on **M** and **S**.
  - This means **T** is a parent of **M** and **S**.

The Bayesian network is as follows (due to limitations, only the edges are described):

$$A \rightarrow T \leftarrow O$$

$$T \rightarrow M, \quad T \rightarrow S$$

Joint Probability Distribution:

Given the structure of the Bayesian network, the joint probability distribution is expressed as:

$$P(A, O, T, M, S) = P(A) \times P(O) \times P(T|A, O) \times P(M|T) \times P(S|T)$$

Checking the validity of the statements:

(3.1) $O \perp A$**: Answer: Correct.**

- In the Bayesian network, O and A have no edge between them and are independent parents. They also do not have a common child given.
- Therefore, O and A are independent.

(3.2) $O \perp A|M$**: Answer: Incorrect.**

- When we condition on M, which is a descendant of T, a dependency is created between O and A.
- This is because both O and A influence T, and T influences M.
- In this case, knowing M gives us information about T, which in turn can create a dependency between O and A.

(3.3) $S \perp M$**: Answer: Incorrect.**

- S and M are both direct children of T.
- Without conditioning on T, S and M are dependent through T.

(3.4) $S \perp M|O$**: Answer: Incorrect.**

- Conditioning on O does not block the dependency path between S and M.
- The dependency path from S to M through T still exists.
- To make S and M independent, we must condition on T, not O.

Summary:

- (3.1) Correct: O and A are independent.
- (3.2) Incorrect: O and A become dependent given M.
- (3.3) Incorrect: S and M are dependent.
- (3.4) Incorrect: Conditioning on O does not make S and M independent.

1.2 Subsection 2

To answer this question, we first need to consider the structure and dependencies present in the Markov graph. A Markov graph represents a conditional dependency structure where each node (variable) is dependent only on its neighboring nodes. Therefore, using the rules of conditional independence in a Markov graph, we can determine the correctness of each given statement.

1. **I** $\perp$ **F**: This statement claims that I and F are independent. According to the graph structure, I and F are connected through G, so they are not independent. This statement is incorrect.
2. **B** $\perp$ **I** — **F**: This statement claims that B and I are independent given F. Since B and I are connected by a chain of nodes in the graph and conditioning on F does not block this chain, this statement is incorrect.
3. **A** $\perp$ **G** — **C, E**: This statement claims that A and G are independent given C and E. In the graph, A and G are connected by a chain of nodes, but with C and E given, there is no direct path between them. Therefore, this statement is incorrect.
4. **P(B—C, D, F) = P(B—C, D)**: This statement claims that the probability of B given C, D, and F is equal to the probability of B given C and D. According to the graph structure and the fact that F has no influence on B in the presence of C and D, this conditional independence holds. Therefore, this statement is correct.

Conclusion:

- Statement 1 is incorrect.
- Statement 2 is incorrect.
- Statement 3 is incorrect.
- Statement 4 is correct.

1.3 Subsection 3

1. **Joint distribution of variables according to the Bayesian graph:** To write the joint distribution according to the given Bayesian graph, we use the factorized form. If we analyze this Bayesian graph based on its nodes and edges, the joint distribution is factorized as follows:

$$P(X_1, X_2, X_3, X_4, X_5, X_6, X_7) = P(X_1)P(X_2|X_1)P(X_3|X_1)P(X_4|X_2)P(X_5|X_2, X_3)P(X_6|X_5)P(X_7|X_6)$$

2. **Markov blanket for variable** X_6**:** The Markov blanket of a variable is the set of variables that, when conditioned upon, makes the variable independent of all other variables in the graph. For X_6, the Markov blanket includes its parents (X_5), its children (X_7), and the parents of its children (here, variable X_4). Thus, the Markov blanket is $\{X_4, X_5, X_7\}$.

3. **Drawing the Markov graph corresponding to the Bayesian graph:** The edges will be the same as the previous edges but undirected. Other edges will also be added due to the V-structure.

$$X2 - X3$$

$$X4 - X6$$

4. **Is the moralized graph from part 3 a perfect i-map for the Bayesian graph? Why?** No, because some independencies that existed in the original graph do not exist in the converted graph. For example, in a V-structure, if we have the child, the two parents will become dependent, but this is not the case here. For instance, we did not have this type of independence.

5. **Is the moralized graph from part 3 chordal? Why?** Yes, it is chordal because in every cycle of length at least 4, we have an edge between two non-adjacent vertices. In other words, the graph is triangulated, so it is chordal.

Definition of a Chordal Graph

A graph is called chordal (also known as an induced acyclic graph or a graph without holes) if every cycle of length 4 or more has a chord, which is an edge connecting two non-consecutive vertices in the cycle. In other words, in a chordal graph, every cycle of at least four nodes must have a shortcut (chord) that divides the cycle into shorter paths.

Key feature of chordal graphs: The main feature of chordal graphs is that they do not contain long cycles without chords, or in other words, chordless cycles. For example:

- A cycle of length 4 (four nodes connected consecutively by edges) must have at least one additional edge between two non-consecutive nodes to be considered a chordal graph.
- If a graph contains a cycle of length 5 or more with no additional edges, this graph is not chordal.

Why is the given graph in the question not chordal? In the question, the graph that is constructed as the moralized graph of the Bayesian network may contain cycles where there is no chord. For example, if there is a cycle of length 4 or more in the Markov graph and this cycle has no additional edges between its non-consecutive nodes, this violates the chordal property.

Simple Example: Suppose we have a simple cycle with four nodes A, B, C, and D, connected as follows:

$$A - B - C - D - A$$

In this cycle, there is no edge between A and C or between B and D. Therefore, this cycle of length 4 is chordless, which makes the graph non-chordal. If we want to make this graph chordal, we must add one of the edges A to C or B to D.

1.4 Subsection 4

1. **If G is a Bayesian graph:** A Bayesian graph represents the joint probability distribution over a set of variables and can show specific conditional independencies. If G is a perfect i-map, it means that all conditional independencies in the distribution are precisely represented by the structure of the Bayesian graph G. If we remove an edge from G and create a new graph G', some new conditional independencies may be created in G' that did not exist in G before. Therefore, G' cannot be a perfect i-map for the same list of independencies L.

2. **If G is a Markov graph:** A Markov graph also represents a list of independencies conditionally. In this case, too, if G is a perfect i-map for the list of independencies L, by removing an edge from G and creating G', new independencies may be created that do not correctly represent the list of independencies L. Consequently, G' cannot be a perfect i-map for L.

1.5 Subsection 5

To estimate the distribution $p(z_1|x_1, x_2)$ using Laplace approximation, we follow these steps:

1. **Write the joint probability:** The joint probability can be expressed as:

$$p(z_1, x_1, x_2) = p(z_1)p(x_1|z_1)p(x_2|z_1)$$

Given the distributions:

$$p(z_1) = N(0, 1), \quad p(x_i|z_1) = N(x_i|z_1, \sigma_i^2) \quad \text{for } i = 1, 2$$

2. **Calculate the log posterior (up to a constant):**

$$\ln p(z_1|x_1, x_2) = \ln p(z_1) + \ln p(x_1|z_1) + \ln p(x_2|z_1) + \text{const}$$

By substituting the normal distributions and simplifying:

$$-\ln p(z_1|x_1, x_2) = \frac{1}{2}z_1^2 + \frac{1}{2\sigma_1^2}(x_1 - z_1)^2 + \frac{1}{2\sigma_2^2}(x_2 - z_1)^2 + \text{const}$$

3. **Find the mode Z by differentiation:** We calculate the derivative with respect to z_1 and set it to zero:

$$\frac{d}{dz_1}\left(\frac{1}{2}z_1^2 + \frac{1}{2\sigma_1^2}(x_1 - z_1)^2 + \frac{1}{2\sigma_2^2}(x_2 - z_1)^2\right) = 0$$

Simplifying the derivative:

$$z_1 - \frac{(x_1 - z_1)}{\sigma_1^2} - \frac{(x_2 - z_1)}{\sigma_2^2} = 0$$

Rewrite and solve for $\hat{z}_1$:

$$z_1\left(1 + \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right) = \frac{x_1}{\sigma_1^2} + \frac{x_2}{\sigma_2^2}$$

$$\hat{z}_1 = \frac{\frac{x_1}{\sigma_1^2} + \frac{x_2}{\sigma_2^2}}{1 + \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}$$

4. **Calculate the variance $\hat{\sigma}^2$:** The second derivative of the negative log posterior gives the inverse of the variance:

$$\frac{d^2}{dz_1^2}(-\ln p) = 1 + \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}$$

$$\hat{\sigma}^2 = \left(1 + \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)^{-1}$$

5. **Write the approximate posterior distribution:** The approximate posterior distribution is a normal distribution with mean $\hat{z}_1$ and variance $\hat{\sigma}^2$:

$$q(z_1) = N(z_1|\hat{z}_1, \hat{\sigma}^2)$$

Final Answer:

The normal approximation is as follows:

$$q(z_1) = N\left(z_1 \middle| \frac{\frac{x_1}{\sigma_1^2} + \frac{x_2}{\sigma_2^2}}{1 + \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}}, \left(1 + \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}\right)^{-1}\right)$$

2 Part 2 - VAE

2.1 Question 1

In Variational Autoencoder (VAE) models, direct computation of the data likelihood function, $\log p(x)$, is very difficult due to computational complexity. To solve this problem, we use the Evidence Lower Bound (ELBO), which is easier to compute. Our goal is to maximize the ELBO, which indirectly helps to maximize $\log p(x)$.

ELBO Definition

$$\text{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x)||p(z))$$

- $\mathbb{E}_{q(z|x)}[\log p(x|z)]$: The data reconstruction term, indicating the quality of reconstruction of X from Z.
- $\text{KL}(q(z|x)||p(z))$: The Kullback-Leibler divergence between the approximate posterior distribution $q(z|x)$ and the prior distribution $p(z)$.

Proof of the relationship between $p(x)$ and ELBO

Our goal is to show this relationship:

$$\log p(x) = \text{ELBO} + \text{KL}(q(z|x)||p(z|x))$$

where $p(z|x)$ is the true posterior distribution and $q(z|x)$ is the approximate posterior distribution.

Proof Steps

1. Start from the definition of $\log p(x)$:

$$\log p(x) = \log \int p(x, z) dz$$

2. Introduce the distribution $q(z|x)$:

$$\log p(x) = \log \int q(z|x) \frac{p(x, z)}{q(z|x)} dz$$

3. Use Jensen's inequality:

$$\log p(x) \geq \int q(z|x) \log \left(\frac{p(x, z)}{q(z|x)} \right) dz$$

4. Split $p(x, z)$ into two parts:

$$\log p(x, z) = \log p(x|z) + \log p(z)$$

5. Substitute and simplify:

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z) + \log p(z) - \log q(z|x)]$$

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x)||p(z))$$

This is the definition of ELBO.

$$\text{ELBO} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x)||p(z))$$

6. Define the difference between $\log p(x)$ and ELBO:

$$\log p(x) - \text{ELBO} = \text{KL}(q(z|x)||p(z|x))$$

Because:

$$\text{KL}(q(z|x)||p(z|x)) = \log p(x) - \mathbb{E}_{q(z|x)}[\log p(x|z) + \log p(z) - \log q(z|x)]$$

which after simplification is equal to:

$$\text{KL}(q(z|x)||p(z|x)) = \log p(x) - \text{ELBO}$$

Conclusion

Final Relationship:

$$\log p(x) = \text{ELBO} + \text{KL}(q(z|x)||p(z|x))$$

Interpretation:

- $\text{KL}(q(z|x)||p(z|x))$ is always non-negative.
- Therefore, ELBO is a lower bound for $\log p(x)$.
- By maximizing ELBO, we indirectly minimize $\text{KL}(q(z|x)||p(z|x))$.
- This causes the approximate distribution $q(z|x)$ to become closer to the true posterior distribution $p(z|x)$.

2.2 Question 2

In training a Variational Autoencoder (VAE), our goal is to maximize the Evidence Lower Bound (ELBO). This involves sampling from the approximate posterior distribution $q(z|x)$ and calculating gradients with respect to the model parameters. However, direct sampling from $q(z|x)$ introduces stochasticity into the computational graph, which makes backpropagation impossible because we cannot calculate gradients through random sampling operations.

To solve this problem, we use the reparameterization trick. This technique converts the random sampling operation into a deterministic one, which allows gradients to be calculated through backpropagation.

Reparameterization Trick

1. **Transforming Sampling:** Instead of sampling Z directly from $q(z|x)$, we express it as a deterministic function of X and a noise variable ϵ:

   $$z = \mu(x) + \sigma(x) \odot \epsilon$$

   where:

   - $\mu(x)$ and $\sigma(x)$ are the outputs of the encoder network (functions of X).
   - $\epsilon \sim N(0, I)$ is a random noise vector sampled from a standard normal distribution.
   - $\odot$ denotes element-wise multiplication.

2. **Deterministic Function:** The transformation $z = g(\epsilon, x)$ is now a deterministic function with respect to the model parameters (embedded in $\mu(x)$ and $\sigma(x)$). This allows us to calculate the gradients of the ELBO with respect to the parameters using standard backpropagation.

3. **ELBO Calculation:** The ELBO is as follows:

   $$\mathcal{L}(x) = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x)||p(z))$$

4. With the reparameterization trick, the expectation can be approximated as follows:

   $$\mathcal{L}(x) \approx \frac{1}{L} \sum_{l=1}^{L} [\log p(x|z^{(l)})] - \text{KL}(q(z|x)||p(z))$$

   where $z^{(l)} = \mu(x) + \sigma(x) \odot \epsilon^{(l)}$ and L is the number of samples.

2.3 Section C

The model was constructed according to the problem statement as follows:

```python
class VariationalAutoencoder(nn.Module):
    def __init__(self, hidden_dim=4096, latent_dim=32):
        super(VariationalAutoencoder, self).__init__()

        self.encoder_net = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=32, kernel_size=4, stride=2, padding=1),
            # nn.BatchNorm2d(32),
            nn.LeakyReLU(0.2),
            # nn.Dropout(0.2),

            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.2),

            nn.Conv2d(in_channels=64, out_channels=128, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),

            nn.Conv2d(in_channels=128, out_channels=256, kernel_size=4, stride=2, padding=1),
            # nn.BatchNorm2d(256),
            nn.LeakyReLU(0.2),
            # nn.Dropout(0.3),

            nn.Flatten(),
            nn.Linear(in_features=256*8*8, out_features=hidden_dim),
            # nn.BatchNorm1d(hidden_dim),
            nn.LeakyReLU(0.2),
            # nn.Dropout(0.3),
        )

        self.mean_fc = nn.Linear(hidden_dim, latent_dim)
        self.logvar_fc = nn.Linear(hidden_dim, latent_dim)

        self.decoder_input_fc = nn.Linear(latent_dim, hidden_dim)
        self.decoder_net = nn.Sequential(
            # nn.BatchNorm1d(hidden_dim),
            nn.LeakyReLU(0.2),
            nn.Linear(hidden_dim, 256*8*8),
            nn.LeakyReLU(0.2),
            # nn.Dropout(0.3),

            nn.Unflatten(dim=1, unflattened_size=(256, 8, 8)),

            nn.ConvTranspose2d(in_channels=256, out_channels=128, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.3),

            nn.ConvTranspose2d(in_channels=128, out_channels=64, kernel_size=4, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.LeakyReLU(0.2),
            nn.Dropout(0.2),

            nn.ConvTranspose2d(in_channels=64, out_channels=32, kernel_size=4, stride=2, padding=1),
            # nn.BatchNorm2d(32),
            nn.LeakyReLU(0.2),
            # nn.Dropout(0.2),

            nn.ConvTranspose2d(in_channels=32, out_channels=3, kernel_size=4, stride=2, padding=1),
            nn.Tanh(),
        )
```

Figure 1: VAE Model Implementation

Since a lot of processing power was not available, we slightly modified the model to achieve better results.

Initial Training Information

```
num_epochs = 1000
initial_learning_rate = 0.0005

vae_model = VariationalAutoencoder().to(device)
optimizer = torch.optim.Adam(vae_model.parameters(), lr=initial_learning_rate)

train_total_losses = []
validation_losses = []
train_reconstruction_losses = []
train_kl_divergence_losses = []
```

Figure 2: Training Setup

The structure of the model's training code was such that it only updated the model based on its corresponding loss function:

```python
for epoch in range(num_epochs):
    vae_model.train()
    epoch_train_loss = 0
    epoch_reconstruction_loss = 0
    epoch_kl_divergence_loss = 0

    for batch_index, (batch_data, _) in enumerate(training_data_loader):
        batch_data = batch_data.to(device)
        optimizer.zero_grad()

        reconstructed_batch, latent_mu, latent_logvar = vae_model(batch_data)
        total_loss, reconstruction_loss, kl_divergence_loss = vae_loss_function(
            reconstructed_batch, batch_data, latent_mu, latent_logvar
        )

        total_loss.backward()
        optimizer.step()

        epoch_train_loss += total_loss.item()
        epoch_reconstruction_loss += reconstruction_loss.item()
        epoch_kl_divergence_loss += kl_divergence_loss.item()

    train_total_losses.append(epoch_train_loss / len(training_data_loader.dataset))
    train_reconstruction_losses.append(epoch_reconstruction_loss / len(training_data_loader.dataset))
    train_kl_divergence_losses.append(epoch_kl_divergence_loss / len(training_data_loader.dataset))

    vae_model.eval()
    epoch_validation_loss = 0

    with torch.no_grad():
        for val_data, _ in validation_data_loader:
            val_data = val_data.to(device)
            recon_val_batch, val_mu, val_logvar = vae_model(val_data)
            val_loss, _, _ = vae_loss_function(recon_val_batch, val_data, val_mu, val_logvar)
            epoch_validation_loss += val_loss.item()

    validation_losses.append(epoch_validation_loss / len(validation_data_loader.dataset))

    print(f'Epoch {epoch}, Train Loss: {train_total_losses[-1]:.4f}, Validation Loss: {validation_losses[-1]:.4f}')
```

Figure 3: Training Loop

In this code, the training and evaluation process of the Variational Autoencoder (VAE) model is fully implemented. The goal of this process is to train the model to reconstruct data in a way that both reduces the reconstruction error and learns a regularized and compressed latent space.

Status of the Final Epochs in the Model

```
Epoch 961, Train Loss: 1712.5202, Validation Loss: 1949.6487
Epoch 962, Train Loss: 1692.0791, Validation Loss: 1967.5531
Epoch 963, Train Loss: 1743.7827, Validation Loss: 1940.3621
Epoch 964, Train Loss: 1691.6240, Validation Loss: 1938.0363
Epoch 965, Train Loss: 1691.2165, Validation Loss: 1998.0003
Epoch 966, Train Loss: 1700.1985, Validation Loss: 1925.8239
Epoch 967, Train Loss: 1697.4328, Validation Loss: 1936.5205
Epoch 968, Train Loss: 1708.6008, Validation Loss: 1965.4723
Epoch 969, Train Loss: 1698.2088, Validation Loss: 1954.9462
Epoch 970, Train Loss: 1711.8245, Validation Loss: 1938.1280
Epoch 971, Train Loss: 1718.4156, Validation Loss: 1971.8726
Epoch 972, Train Loss: 1709.8836, Validation Loss: 1942.8837
Epoch 973, Train Loss: 1674.0670, Validation Loss: 1940.3383
Epoch 974, Train Loss: 1706.9505, Validation Loss: 1936.4837
Epoch 975, Train Loss: 1735.8962, Validation Loss: 1945.9228
Epoch 976, Train Loss: 1722.3701, Validation Loss: 1994.9492
Epoch 977, Train Loss: 1673.2198, Validation Loss: 1936.0148
Epoch 978, Train Loss: 1710.9364, Validation Loss: 1973.5153
Epoch 979, Train Loss: 1673.2449, Validation Loss: 1923.6583
Epoch 980, Train Loss: 1699.6863, Validation Loss: 1937.6884
Epoch 981, Train Loss: 1720.3276, Validation Loss: 1948.9264
Epoch 982, Train Loss: 1685.5296, Validation Loss: 1943.0112
Epoch 983, Train Loss: 1696.6006, Validation Loss: 1949.8939
Epoch 984, Train Loss: 1707.3807, Validation Loss: 1914.1065
Epoch 985, Train Loss: 1679.9964, Validation Loss: 1931.7288
Epoch 986, Train Loss: 1714.7194, Validation Loss: 1935.9698
Epoch 987, Train Loss: 1735.0827, Validation Loss: 1956.1084
Epoch 988, Train Loss: 1766.4708, Validation Loss: 1953.9749
Epoch 989, Train Loss: 1719.3289, Validation Loss: 1951.0376
Epoch 990, Train Loss: 1709.2564, Validation Loss: 1950.4423
Epoch 991, Train Loss: 1734.2256, Validation Loss: 1919.4284
Epoch 992, Train Loss: 1716.5299, Validation Loss: 1941.5617
Epoch 993, Train Loss: 1684.3253, Validation Loss: 1943.4446
Epoch 994, Train Loss: 1705.7880, Validation Loss: 2025.8981
Epoch 995, Train Loss: 1712.1634, Validation Loss: 1946.8408
Epoch 996, Train Loss: 1732.8812, Validation Loss: 1976.8728
Epoch 997, Train Loss: 1707.9629, Validation Loss: 1936.9383
Epoch 998, Train Loss: 1709.4926, Validation Loss: 1980.3244
Epoch 999, Train Loss: 1721.8064, Validation Loss: 1955.0225
```

Figure 4: Final Epochs Training and Validation Loss

Model Loss Value

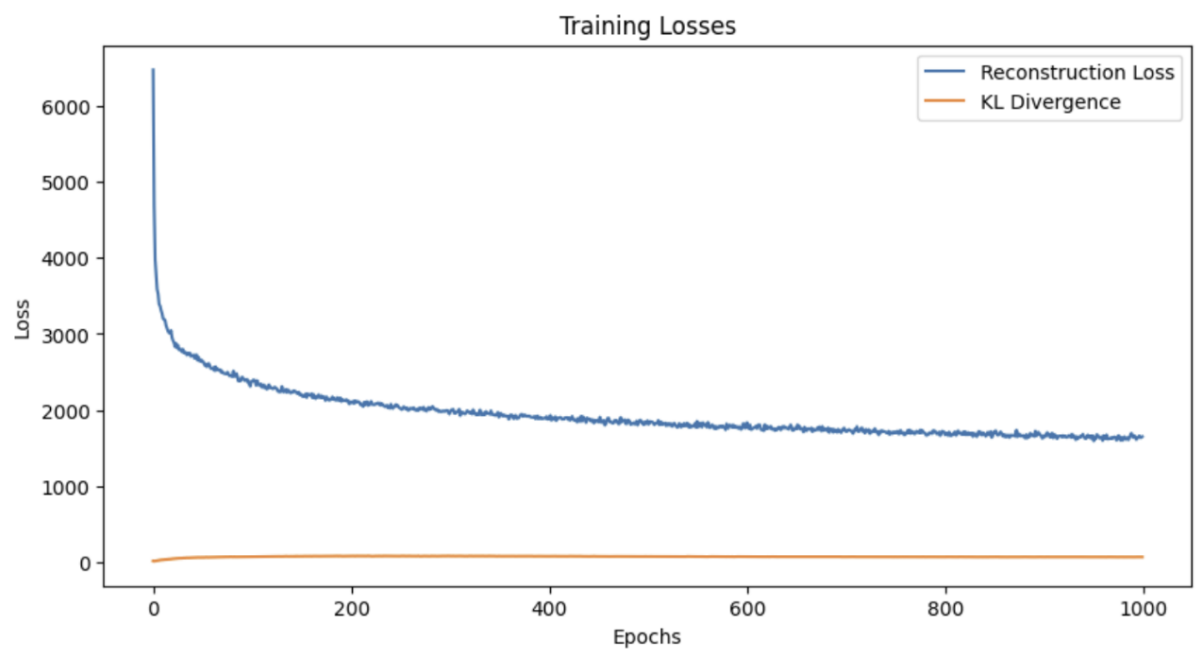

Figure 5: Training Losses (Reconstruction vs. KL Divergence)

In this part, after much effort, we succeeded in gradually reducing the model's cost.

In previous tests, the model quickly overfit on the training images due to the lack of normalization terms in the model's layers. By adding normalization terms, these problems were mostly resolved, resulting in blurrier images.

Because it generally takes a lot of time and data to be able to generate a complete and detailed image for each point in the Latent Space, the images are slightly noisy.

As you can see, at the end of the model training on 128x128 images, the loss reached a value of 1700, which is a small difference from the desired loss of 1500 set by the project designer.

2.4 Section D

The images generated by the model, when we input 128x128 images to the model:

Figure 6: Generated faces (sample 1)

```
vae_model.eval()
with torch.no_grad():
    validation_iterator = iter(validation_data_loader)
    sample_data, _ = next(validation_iterator)
    sample_data = sample_data.to(device)

    reconstructed_batch, _, _ = vae_model(sample_data)

    num_images_display = 10

    image_comparison = torch.cat([sample_data[:num_images_display], reconstructed_batch[:num_images_display]])
    image_comparison = image_comparison.cpu()

    grid_image = torchvision.utils.make_grid(image_comparison, nrow=num_images_display)
    display_images(grid_image)

    sample_data, _ = next(validation_iterator)
    sample_data = sample_data.to(device)
    reconstructed_batch, _, _ = vae_model(sample_data)

    num_images_display = 10

    image_comparison = torch.cat([sample_data[:num_images_display], reconstructed_batch[:num_images_display]])
    image_comparison = image_comparison.cpu()

    grid_image = torchvision.utils.make_grid(image_comparison, nrow=num_images_display)
    display_images(grid_image)


with torch.no_grad():
    sample_data, _ = next(validation_iterator)
    sample_data = sample_data.to(device)

    reconstructed_batch, _, _ = vae_model(sample_data)

    num_images_display = 12

    image_comparison = torch.cat([sample_data[:num_images_display], reconstructed_batch[:num_images_display]])
    image_comparison = image_comparison.cpu()

    grid_image = torchvision.utils.make_grid(image_comparison, nrow=num_images_display)
    display_images(grid_image)
```

Figure 7: Generated faces (sample 2)

As we can see, the images are a bit noisy. If we average the images with and without smiles, we will reach these results in the end:

```
vae_model.eval()
with torch.no_grad():
    validation_iterator = iter(validation_data_loader)
    sample_data, _ = next(validation_iterator)
    sample_data = sample_data.to(device)

    reconstructed_batch, _, _ = vae_model(sample_data)

    num_images_display = 10

    image_comparison = torch.cat([sample_data[:num_images_display], reconstructed_batch[:num_images_display]])
    image_comparison = image_comparison.cpu()

    grid_image = torchvision.utils.make_grid(image_comparison, nrow=num_images_display)
    display_images(grid_image)

    sample_data, _ = next(validation_iterator)
    sample_data = sample_data.to(device)
    reconstructed_batch, _, _ = vae_model(sample_data)

    num_images_display = 10

    image_comparison = torch.cat([sample_data[:num_images_display], reconstructed_batch[:num_images_display]])
    image_comparison = image_comparison.cpu()

    grid_image = torchvision.utils.make_grid(image_comparison, nrow=num_images_display)
    display_images(grid_image)


with torch.no_grad():
    sample_data, _ = next(validation_iterator)
    sample_data = sample_data.to(device)

    reconstructed_batch, _, _ = vae_model(sample_data)

    num_images_display = 12

    image_comparison = torch.cat([sample_data[:num_images_display], reconstructed_batch[:num_images_display]])
    image_comparison = image_comparison.cpu()

    grid_image = torchvision.utils.make_grid(image_comparison, nrow=num_images_display)
    display_images(grid_image)
```

Figure 8: Code for displaying image comparisons.

Figure 9: Interpolation between original and reconstructed images.

The code structure was such that we first had to obtain the average of smiling and non-smiling images using the 'calculate$_{l}atent_{m}eans$' $function. The we took a weighted average of the min 6 places and obtained the output. As can be$

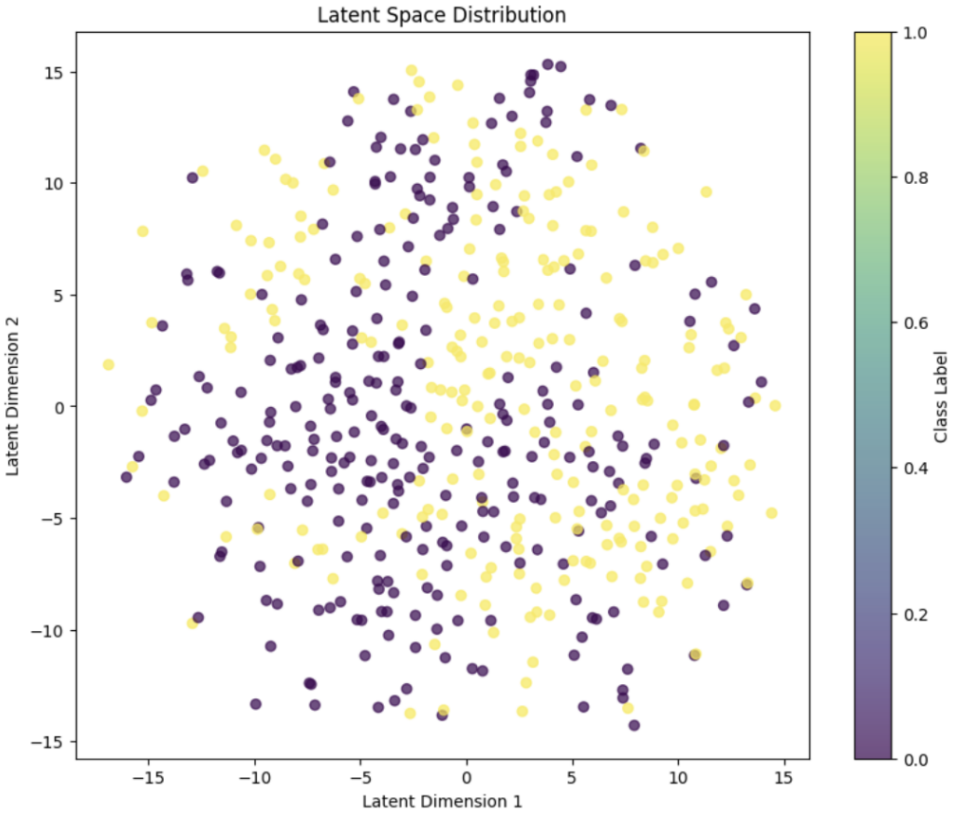

Figure 10: Latent Space Distribution.

The mean variance in each dimension is shown:

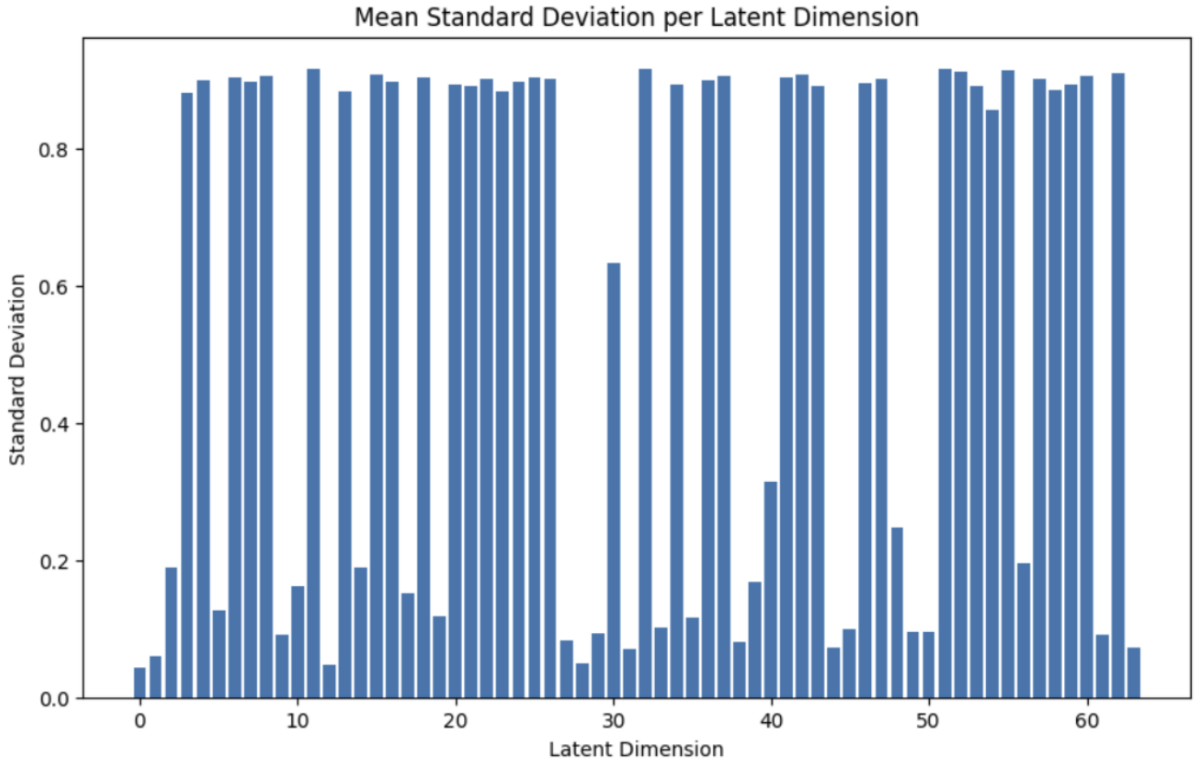

Figure 11: Mean Standard Deviation per Latent Dimension.

```
def calculate_latent_means(model, data_loader):
    model.eval()
    latent_means_list, label_collection = [], []
    with torch.no_grad():
        for batch_data, batch_labels in data_loader:
            batch_data = batch_data.to(device)
            encoded_features = model.encoder(batch_data)
            latent_mean = model.mean_fc(encoded_features)
            latent_means_list.append(latent_mean.cpu())
            label_collection.append(batch_labels)
    concatenated_latents = torch.cat(latent_means_list)
    concatenated_labels = torch.cat(label_collection)

    mean_smile_latent = concatenated_latents[concatenated_labels == 0].mean(dim=0)
    mean_nonsmile_latent = concatenated_latents[concatenated_labels == 1].mean(dim=0)

    return mean_smile_latent, mean_nonsmile_latent
```

Figure 12: Code for calculating latent means.

```
mean_smile_latent, mean_nonsmile_latent = calculate_latent_means(vae_model, training_data_loader)
latent_direction_vector = (mean_smile_latent - mean_nonsmile_latent).to(device)

validation_data_batch = next(iter(validation_data_loader))
sample_data, sample_labels = validation_data_batch
sample_data = sample_data.to(device)

encoded_sample_data = vae_model.encoder(sample_data)
latent_means_batch = vae_model.mean_fc(encoded_sample_data)

selected_sample_index = 0
z_sample = latent_means_batch[selected_sample_index]

alpha_range = np.linspace(-3, 3, 7)
interpolated_images = []

with torch.no_grad():
    for alpha in alpha_range:
        adjusted_latent = z_sample + alpha * latent_direction_vector
        decoder_input_data = vae_model.decoder_input(adjusted_latent)
        decoded_image = vae_model.decoder(decoder_input_data.unsqueeze(0))
        interpolated_images.append(decoded_image.cpu())

grid_image = torchvision.utils.make_grid(torch.cat(interpolated_images), nrow=len(alpha_range))
display_images(grid_image)
```

Figure 13: Code for latent space interpolation.

This code provides a function to plot the mean standard deviation for each dimension in the latent space of a VAE model.

1. First, it sets the VAE model to evaluation mode and creates a variable to store the standard deviations.
2. Then, through the input data from the DataLoader, without calculating gradients, it receives images and feeds them into the model.
3. The model generates the mean and log variance for the latent codes. The standard deviation is calculated for each latent dimension and added to the list.
4. The standard deviations of all samples are combined and averaged.
5. Finally, a bar chart of the mean standard deviation for each dimension in the latent space is displayed.

3 Part 2 - Theory

3.1 Question 1

Introduction

In Variational Autoencoder (VAE) models, our goal is to learn a latent distribution $q(z|x)$ that can reconstruct the real data X well. In β-VAE, the emphasis is on discovering disentangled latent factors, such that each variable in the latent representation is sensitive to only one specific factor and remains relatively constant with respect to other factors.

Formulating the Constrained Optimization Problem

To achieve this goal, we want to maximize the probability of generating real data while keeping the distance between the true latent distribution and its approximation small (e.g., less than a small constant δ). This problem can be formulated as a constrained optimization problem.

Objective: Our goal is to maximize the following function:

$$\max_{q(z|x)} \mathbb{E}_{q(z|x)}[\log p(x|z)]$$

Constraint:

$$\mathrm{KL}(q(z|x)||p(z)) \leq \delta$$

where KL Divergence is between the approximate posterior distribution $q(z|x)$ and the prior distribution $p(z)$.

Using the Lagrangian and KKT Conditions

To solve the constrained optimization problem, we use the method of Lagrange multipliers and the Karush-Kuhn-Tucker (KKT) conditions.

1. Constructing the Lagrangian Function: The Lagrangian function $\mathcal{L}$ is defined as follows:

$$\mathcal{L} = -\mathbb{E}_{q(z|x)}[\log p(x|z)] + \lambda(\mathrm{KL}(q(z|x)||p(z)) - \delta)$$

Note that a negative sign is placed in front of $\mathbb{E}_{q(z|x)}[\log p(x|z)]$ because we want to minimize this term (the opposite of maximization).

2. KKT Conditions: The KKT conditions for this problem are:

- **(a) Stationarity:** $\frac{\delta \mathcal{L}}{\delta q(z|x)} = 0$
- **(b) Complementary Slackness:** $\lambda(\mathrm{KL}(q(z|x)||p(z)) - \delta) = 0$
- **(c) Dual Feasibility:** $\lambda \geq 0$
- **(d) Primal Feasibility:** $\mathrm{KL}(q(z|x)||p(z)) - \delta \leq 0$

3. Calculating the Derivative of the Lagrangian: The derivative of the Lagrangian with respect to $q(z|x)$ is:

$$\frac{\delta \mathcal{L}}{\delta q(z|x)} = -\log p(x|z) + \lambda(\log q(z|x) - \log p(z) + 1)$$

4. Setting the Derivative to Zero: To find $q(z|x)$, we set the derivative to zero:

$$-\log p(x|z) + \lambda(\log q(z|x) - \log p(z) + 1) = 0$$

5. Solving for $q(z|x)$: Rewriting the equation:

$$\lambda(\log q(z|x) - \log p(z)) = \log p(x|z) - \lambda$$

Then:

$$\log q(z|x) = \frac{1}{\lambda} \log p(x|z) + \log p(z) - 1$$

Therefore:

$$q(z|x) \propto p(x|z)^{\frac{1}{\lambda}} p(z)$$

6. Defining β: By defining $\beta = \lambda$, we can simplify the relationship.

7. Final Cost Function: Substituting β into the Lagrangian function:

$$\mathcal{L} = -\mathbb{E}_{q(z|x)}[\log p(x|z)] + \beta(\mathrm{KL}(q(z|x)||p(z)) - \delta)$$

Since δ is a constant and does not affect the optimization process (due to the complementary slackness condition $\lambda(KL - \delta) = 0$), we can ignore it. Thus, the cost function simplifies to:

$$\mathcal{L}_\beta = -\mathbb{E}_{q(z|x)}[\log p(x|z)] + \beta\mathrm{KL}(q(z|x)||p(z))$$

Or, by changing the sign:

$$\mathcal{L}_\beta = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta\mathrm{KL}(q(z|x)||p(z))$$

Using the Lagrangian method and KKT conditions, we have shown that by imposing a constraint on the KL divergence between $q(z|x)$ and $p(z)$, the β-VAE cost function is obtained:

$$\mathcal{L}_\beta = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta\mathrm{KL}(q(z|x)||p(z))$$

Supplementary Explanations

- The coefficient β plays an important role in controlling the balance between data reconstruction and disentanglement of latent factors. Increasing β imposes a greater penalty on the KL divergence, which leads to greater disentanglement but may reduce the quality of reconstruction.
- The Lagrangian method allows us to convert a constrained optimization problem into an unconstrained one that can be solved by minimizing the Lagrangian function.
- The KKT conditions ensure that our optimal solution satisfies the problem constraints and is at a minimum point of the Lagrangian function.

3.2 Theory Question 2

In the β-VAE model, the cost function is defined as follows:

$$\mathcal{L}_\beta = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \beta\mathrm{KL}(q(z|x)||p(z))$$

where:

- $\mathbb{E}_{q(z|x)}[\log p(x|z)]$: The reconstruction term that tries to maximize the probability of reconstructing the data X from the latent variables z.
- $\mathrm{KL}(q(z|x)||p(z))$: The KL divergence between the approximate posterior distribution $q(z|x)$ and the prior distribution $p(z)$, which acts as a regularizer.
- β: A parameter that controls the balance between reconstruction and disentanglement of latent factors.

Decomposition of KL Divergence into Two Parts

The goal is to show that the KL divergence in the β-VAE cost function can be decomposed into two parts:

- **Mutual Information between X and Z**: $I_q(x; z)$
- **KL divergence between the marginal distribution $q(z)$ and the prior $p(z)$**: $\mathrm{KL}(q(z)||p(z))$

Definitions of Required Distributions

- **Observed data distribution**: $p_{data}(x)$
- **Approximate posterior distribution**: $q(z|x)$
- **Marginal distribution**: $q(z) = \int q(z|x)p_{data}(x)dx$

2. Starting from the KL divergence in the cost function:

$$\mathbb{E}_{p_{data}(x)}[\mathrm{KL}(q(z|x)||p(z))] = \int p_{data}(x) \int q(z|x) \log \frac{q(z|x)}{p(z)} dzdx$$

3. Adding and subtracting $q(z)$:

Multiplying and dividing by $q(z)$ inside the logarithm:

$$\log \frac{q(z|x)}{p(z)} = \log \frac{q(z|x)}{q(z)} + \log \frac{q(z)}{p(z)}$$

4. Separating the two parts:

Thus, the KL divergence can be written as:

$$\mathbb{E}_{p_{data}(x)}[\mathrm{KL}(q(z|x)||p(z))] = \mathbb{E}_{p_{data}(x)} \left[\int q(z|x) \log \frac{q(z|x)}{q(z)} dz \right] + \int q(z) \log \frac{q(z)}{p(z)} dz$$

- **Part 1: Mutual Information between X and Z:**

$$I_q(x; z) = \mathbb{E}_{p_{data}(x)}[\mathrm{KL}(q(z|x)||q(z))]$$

- **Part 2: KL divergence between $q(z)$ and $p(z)$:**

$$\mathrm{KL}(q(z)||p(z)) = \int q(z) \log \frac{q(z)}{p(z)} dz$$

Therefore:

$$\mathbb{E}_{p_{data}(x)}[\mathrm{KL}(q(z|x)||p(z))] = I_q(x; z) + \mathrm{KL}(q(z)||p(z))$$

In the paper "Disentangling by Factorising," the authors use this decomposition to improve the trade-off between reconstruction quality and disentanglement of latent factors. In β-VAE, increasing β leads to an increase in both terms $I_q(x; z)$ and $\mathrm{KL}(q(z)||p(z))$. This reduces the mutual information between X and Z, which in turn can degrade reconstruction quality.

Solution in FactorVAE

The authors of FactorVAE propose that instead of penalizing the entire KL divergence, we should only focus on the KL divergence between $q(z)$ and $p(z)$, as this term is responsible for the disentanglement of latent factors.

FactorVAE Cost Function:

$$\mathcal{L}_{\text{FactorVAE}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \mathbb{E}_{p_{data}(x)}[\mathrm{KL}(q(z|x)||p(z))] - \gamma \mathrm{TC}(q(z))$$

where:

- **TC(q(z))**: Total Correlation of the distribution $q(z)$, which is a measure of the statistical dependencies between the latent variables.
- γ: A parameter that adjusts the weight of the total correlation penalty.

The total correlation is defined as:

$$\mathrm{TC}(q(z)) = \mathrm{KL}\left(q(z)\left\|\prod_{j=1}^{d} q(z_j)\right.\right)$$

This term measures how much the distribution $q(z)$ differs from the product of its marginal distributions $q(z_j)$. Penalizing this term encourages the distribution $q(z)$ to approach an independent distribution, which helps in disentangling the latent factors. By using the KL divergence decomposition, the FactorVAE cost function can be written as:

$$\mathcal{L}_{\text{FactorVAE}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - I_q(x;z) - \mathrm{KL}(q(z)||p(z)) - \gamma \mathrm{TC}(q(z))$$

However, since TC(q(z)) is actually a part of $\mathrm{KL}(q(z)||p(z))$, we can directly penalize TC(q(z)) without harming $I_q(x;z)$.

Main Advantage

With this method, FactorVAE can improve the disentanglement of latent factors without significantly reducing the reconstruction quality, as the mutual information between X and Z is preserved.

Summary

- In β-VAE, penalizing the entire KL divergence leads to a reduction in mutual information $I_q(x;z)$, which can harm reconstruction quality.
- In FactorVAE, by decomposing the KL divergence and penalizing the Total Correlation term TC(q(z)), we can reduce the dependencies between latent variables and improve disentanglement, while preserving mutual information.
- Conclusion: FactorVAE offers a better trade-off between reconstruction quality and disentanglement of latent factors.

3.3 Theory Question 3

Introduction

In the FactorVAE objective function, the KL divergence between the distributions $q(z)$ and $\bar{q}(z)$ (where $\bar{q}(z) = \prod_j q(z_j)$) is present.

$$\mathcal{L}_{\text{FactorVAE}} = \frac{1}{N}\sum_{i=1}^{N}\left[\mathbb{E}_{q(z|x^{(i)})}[\log p(x^{(i)}|z)] - \mathrm{KL}(q(z|x^{(i)})||p(z))\right] - \gamma \mathrm{KL}(q(z)||\bar{q}(z))$$

Direct computation of the distributions $q(z)$ and $\bar{q}(z)$ requires complex and computationally expensive integrations, because $q(z)$ is a mixture (average) of the distributions $q(z|x)$ over all data:

$$q(z) = \int q(z|x)p_{\text{data}}(x)dx$$

Therefore, the exact calculation of these distributions is practically impossible.

Proposed Methods for Approximation of $q(z)$ and $\bar{q}(z)$

The authors of the paper "Disentangling by Factorising" have proposed several methods for estimating $q(z)$ and $\bar{q}(z)$:

1. **Using Direct Sampling from $q(z)$:**
   - By randomly selecting a data sample $x^{(i)}$ and then sampling from $q(z|x^{(i)})$, one can obtain samples from $q(z)$.
2. **Using Batch-based Approximations for $q(z)$:**

- Using the samples available in a batch, approximations for $q(z)$ can be obtained as $\bar{q}(z) = \frac{1}{B}\sum_{i=1}^{B} q(z|x^{(i)})$.
- However, this method is not practically efficient due to the need for large batches and heavy computations.

3. **Using the Dimension Shuffling Trick to Estimate $\bar{q}(z)$:**
   - By randomly shuffling the values of each dimension z among the samples of a batch, samples from $\bar{q}(z)$ can be obtained.
   - This method breaks the dependency between dimensions, and the resulting distribution is approximately equal to $\bar{q}(z)$.
4. **Using the Density Ratio Trick and Training a Discriminator Network:**
   - Using samples from $q(z)$ and $\bar{q}(z)$, a discriminator can be trained to distinguish between these two distributions.
   - Then, using the discriminator's output, the KL divergence can be estimated.

Selected Method and its Explanation

The method chosen by the authors is the use of the density ratio trick and training a discriminator.

Explanation of the Method:

1. **Sampling from $q(z)$:**
   - First, a batch of data $x^{(i)}$ is selected.
   - From each $x^{(i)}$, a sample of z is obtained using $q(z|x^{(i)})$.
2. **Creating samples of $\bar{q}(z)$:**
   - In the same batch, for each dimension j of z, the values are randomly shuffled among the samples.
   - This action breaks the correlation between dimensions, and the resulting samples are from $\bar{q}(z)$.
3. **Training the Discriminator:**
   - A discriminator network is defined that takes z as input and its output is the probability $D(z)$ that z came from $q(z)$ (as opposed to $\bar{q}(z)$).
   - The goal of the discriminator is to distinguish between samples from $q(z)$ and $\bar{q}(z)$.
4. **Estimating KL divergence using the density ratio:**
   - The outputs of the discriminator are used to estimate the density ratio $\frac{q(z)}{\bar{q}(z)}$. According to the relation:
$$\mathrm{KL}(q(z)||\bar{q}(z)) = \mathbb{E}_{q(z)}\left[\log \frac{q(z)}{\bar{q}(z)}\right] \approx \mathbb{E}_{q(z)}\left[\log \frac{D(z)}{1-D(z)}\right]$$
5. **Updating Parameters:**
   - The VAE parameters are updated using the gradient of the objective function, which includes the estimated KL term.
   - The discriminator parameters are updated using data from $q(z)$ and $\bar{q}(z)$ to perform better discrimination.

Advantages of the Selected Method

- **No need for heavy computations:** By using this method, the need for direct calculation of $q(z)$ and $\bar{q}(z)$ is eliminated.
- **Practical efficiency:** It allows for practical and efficient training of the model.
- **Preservation of gradient flow:** This method allows us to correctly calculate the gradients through the VAE network.

References

[1] Kim, H., & Mnih, A. (2018). Disentangling by Factorising. In *Proceedings of the 35th International Conference on Machine Learning (ICML).*

[2] Chen, T. Q., et al. (2018). Isolating Sources of Disentanglement in Variational Autoencoders. In *Advances in Neural Information Processing Systems (NeurIPS).*

[3] Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. In *Proceedings of the International Conference on Learning Representations (ICLR).*

[4] Higgins, I., Matthey, L., Pal, A., et al. (2017). beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework. In *International Conference on Learning Representations (ICLR).*

[5] Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.

[6] Blei, D. M. (Lecture Notes). Variational Inference. Sections on KL Divergence and Mean Field Variational Inference.

[7] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. Sections on Variational Inference and Gaussian Distributions.